

Diagrama C4 — CliniDesk

Contexto, Containers e Componentes do Sistema NPI

Projeto: CliniDesk — NPI

Versao: v1.0.0

Sprint: Sprint 2 · 2026

Modelo C4 — Visao Arquitetural

O modelo C4 (*Context, Containers, Components, Code*) e uma abordagem hierarquica para documentar a arquitetura de software. Este documento cobre os niveis **N1 (Contexto)**, **N2 (Containers)** e **N3 (Componentes)** do CliniDesk.

N1 — Contexto do Sistema

O contexto define **quem usa** o sistema e **com o que ele se conecta** externamente. O CliniDesk atua como o nucleo digital da recepcao do NPI.

Atores do Sistema

Ator	Tipo	Interacao com o Sistema
Recepcionista	Usuario Primario	Agenda consultas, busca pacientes, envia lembretes e confirma presenca
Gestor	Usuario Secundario	Confere historico de mudancas e valida integridade dos dados
Idoso / Responsavel	Receptor Externo	Recebe mensagem no WhatsApp e lembrete impresso — nao acessa o sistema diretamente

Sistemas Externos

Sistema	Tecnologia	Integracao
WhatsApp Web	Browser redirect + URL encode	Sistema gera mensagem formatada e abre o WhatsApp Web com o chat pronto para envio com um clique
Impressora	window.print() / PDF blob	Geracao de bilhete de consulta para impressao fisica (termica ou jato de tinta) ou exportacao em PDF
MongoDB Atlas	Driver Mongoose via TCP/TLS	Persistencia de todos os dados na nuvem com autenticao segura e conexao criptografada

Diagrama de Contexto (N1)

```
[ Recepcionista ] ----HTTP----> [ CliniDesk – Sistema Web ]  
[ Gestor ] -----> |  
|----> [ MongoDB Atlas ]  
|----> [ WhatsApp Web ]  
|----> [ Impressora ]  
[ Idoso / Responsavel ] <-- notificacao ----+
```

N2 — Containers

O nível de Containers mostra como o sistema é dividido em processos/serviços que se comunicam entre si. Cada container é uma unidade deployável com responsabilidades bem definidas.

FRONTEND

React + Vite — Single Page Application

Porta 5173 | Pasta /Front

Responsabilidades:

- Exibe o Dashboard com tabela de agendamentos, abas de Consultas e Retornos
- Aplica Smart Color-Coding em tempo real: Verde / Amarelo / Vermelho por horário
- Abre ModalNovoAgendamento para criação e edição de consultas
- Dispara requisições HTTP via Axios para a API do backend (porta 5000)
- Interceptor global captura HTTP 401 e redireciona ao login automaticamente
- Gera URL do WhatsApp Web com mensagem pre-formatada para o responsável
- Aciona impressão ou exportação PDF do bilhete de consulta

Stack: React.js v18+ | Vite | Axios | date-fns | lucide-react | Tailwind CSS / Custom CSS

BACKEND

Node.js + Express — API REST

Porta 5000 | Pasta /backend | Entrada: src/server.js

Responsabilidades:

- Valida cookie HttpOnly com JWT em todas as rotas protegidas (authMiddleware)
- Aplica rate-limiting no endpoint de login: max. 5 tentativas / 15 minutos
- Valida dados de entrada via middleware validateAgendamento antes de persistir
- Executa CRUD de Consultas e Retornos: criar, listar, atualizar, cancelar
- Registra lembrete: grava ID do usuário e timestamp do envio no documento
- Gerencia sessão via cookie seguro (HttpOnly + SameSite=Strict + Secure)

Stack: Node.js | Express | Mongoose | jsonwebtoken | bcryptjs | cookie-parser | express-rate-limit | dotenv | cors

BANCO DE DADOS

MongoDB Atlas — NoSQL Document Store

Hospedado na nuvem | Conexao via MONGO_URI no .env

Responsabilidades:

- Armazena documentos de Agendamentos: paciente, responsavel, data, horario, setor, status
- Armazena usuarios: username, hash da senha (bcrypt), nivel de acesso
- Schema flexivel permite adicionar campos sem necessidade de migrations
- Conectado ao backend via Mongoose com validacoes de schema e indices

Stack: MongoDB Atlas | Mongoose ODM

Fluxo de Comunicacao entre Containers

```
Browser (React SPA)
|
| HTTPS + cookie HttpOnly
v
Express API (Node.js) ---- Mongoose/TLS ----> MongoDB Atlas
|
| valida JWT cookie
| retorna 401 se invalido -----> interceptor React redireciona ao login
|
+-----> WhatsApp Web (redirect URL com mensagem encode)
+-----> Impressora (window.print() / PDF Blob)
```

N3 — Componentes Principais

Frontend — Componentes React

Componente	Responsabilidade
Dashboard.jsx	Painel principal: tabela de agendamentos com abas Consultas / Retornos, filtros reativos por data e setor, Smart Color-Coding de horarios (verde/amarelo/vermelho), acoes de check-in e registro de lembrete. Chama buscarConsultas() e puxarRetornosDoBanco() apos toda operacao para manter dados frescos sem F5.
ModalNovoAgendamento.jsx	Formulario de criacao de Consulta e Retorno. Aplica Guard Clauses (Early Return) nas validacoes. Opcoes de setor carregadas via props (sem hardcode). Trava de seguranca no handleSubmit bloqueia agendamento em data/hora passada. Limpa chaves de retorno antes do envio se nao preenchidas (previne erro 422).
App.jsx	Roteamento SPA, estado global de autenticacao, interceptor Axios para HTTP 401.
Login.jsx	Tela de autenticacao com username e senha. Chama POST /api/auth/login.

Backend — Modulos Node.js

Modulo / Arquivo	Responsabilidade
routes/pacienteRoutes.js	Define as rotas REST: GET / POST / PUT / DELETE de consultas e retornos. Rotas de retorno separadas para evitar conflito com rotas dinamicas (:id).
controllers/patientController.js	Logica de negocio: processa requisicoes, aplica regras e persiste via Mongoose. Usa desestruturacao explicita de req.body para prevenir Mass Assignment.
middlewares/authMiddleware.js	Verifica e valida o cookie HttpOnly com o token JWT em todas as rotas protegidas. Retorna HTTP 401 se token ausente, invalido ou expirado.
middlewares/validateAgendamento.js	Valida telefone, data (DD/MM/YYYY), horario (08:00-18:00), setor e campos obrigatorios antes de encaminhar ao controller.
utils/validators.js	Funcoes puras de validacao: isValidNome, isValidTelefone, isValidDataConsulta, isValidHorario. Testadas com Vitest (20 testes unitarios).

utils/formatters.js	Funcoes puras de formatacao: formatarData (DD/MM/YYYY), normalizarNome (capitaliza).
utils/calculators.js	Regras de negocio puras: diasParaConsulta (dias restantes), isHorarioDisponivel (detecta conflitos).
models/Agendamento.js	Schema Mongoose do agendamento com validacoes de campo e suporte a Consulta + Retorno.
models/User.js	Schema Mongoose do usuario com hash de senha via bcryptjs.

Arquitetura de Dados — Tratamento de Fuso Horário

Um dos desafios tecnicos do projeto foi o tratamento de fuso horario no MongoDB. Por padrao, o MongoDB armazena datas no formato ISO com timezone (2026-05-28T00:00:00.000Z). Para evitar que consultas sumissem do calendario apos conversao automatica do fuso, foi implementado um pipeline de tratamento de dados de duas vias:

1. Envio (Insercao):

O formulario intercepta o objeto Date gerado no frontend e o transforma na string linear padronizada yyyy-MM-dd antes de submeter ao servidor.

2. Recebimento (Renderizacao e Filtros):

O Dashboard quebra a string ISO trazida pelo backend, removendo o sufixo de timezone:

```
// Higienizacao que remove residuos de fuso horario (T00:00...)
const dataConsultaLimpa = consulta.data.includes('T')
? consulta.data.split('T')[0]
: consulta.data;
```

Bugs Criticos Resolvidos (Sprint 2)

Bug	Causa	Solucao
Erro 404 no cancelamento	Rota dinamica /consultas/:id capturava a string 'retornos' antes de chegar na rota correta	Rotas de retorno movidas para /consultas/retornos/:id, antes das rotas dinamicas no arquivo de rotas
Erro 422 Unprocessable Entity	Envio de chaves de retorno vazias ou nulas quando o usuario nao preenchia o retorno opcional	Chaves de retorno sao deletadas do objeto antes do envio se nao preenchidas: delete dadosProntos.dataRetorno

Cannot read properties (React)	Banco retornava itens undefined em mapeamentos de array causando quebra do componente	Adicao de validacoes defensivas: consulta && consulta._id antes de todo mapeamento
---------------------------------------	---	--